

# BILGISAYAR PROGRAMLAMA



# Genel Bakış...

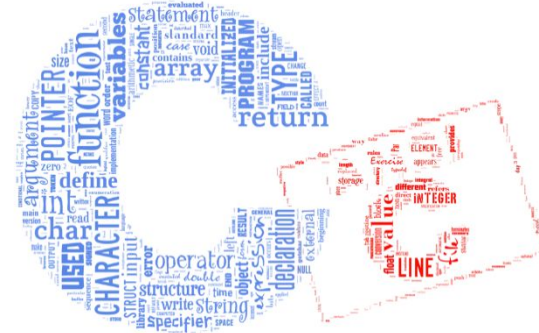
2

- C Programlama Dili Tarihçesi
- C Programlama Dili Tercih Nedeni
- İlk C Programı
- C Kodlarının Temel Özellikleri
- İlk C Programı
- C Programlama Dili Elemanları

# C Programlama Dilinin Tarihçesi

3

- C programlama dili, **geniş amaçlı, orta seviyeli, yapısal** bir programlama dilidir.
- C, temelde iki eski dile dayanır : BCPL ve B.
- BCPL, 1967 yılında Martin Richards tarafından işletim sistemleri ve derleyiciler yazmak için geliştirilmiştir.
- C programlama dili 1972 yılında Bell Laboratuvarlarında Dennis Ritchie tarafından geliştirilmiştir.
- C programlama dili, UNIX işletim sisteminin kodlama ve geliştirilmesinde kullanılarak popülerliğini arttırmıştır.



# C Programlama Dilinin Tarihçesi (devam...)

- 1970'lerin sonunda C, şu anda geleneksel C olarak bilinen haline geldi. 1978 yılında Kernighan ve Ritchie tarafından yazılan, «The C Programming Language» adlı kitabın yayınlanmasından sonra, C'ye olan ilgi artmıştır.
- ANSI C, C programlama dilinin, 1989 yılında Amerika'da «American National Standards Institute (ANSI)» ve tüm dünyada «International Standards Organization (ISO)» tarafından standart hale getirilmiş sürümüdür.
- ANSI C standardı, 1989 yılında onaylanmış, 1999 yılında gözden geçirilmiş ve Mart 2000'de C99: ISO/IEC 9899:1999 standardı **Standart C** olarak kabul edilmiştir

# C Programlama Dili Tercih Nedeni

5

- C, en **popüler** dillerdin birisidir.
- C, güçlü ve **esnek** bir dildir. C ile işletim sistemi veya derleyici yazabilir, kelime işlemciler oluşturabilir veya grafikler çizebilirsiniz.
- C, yazılım geliştirme ortamları oldukça fazladır.
- C, özel komut ve **veri tipi tanımlamasına** izin verir.
- C, **taşınabilir** bir dildir.
- C, gelişimini tamamlamış ve **standardı oluşmuş** bir dildir.
- C, **yapısal** bir dildir. C kodları fonksiyon olarak adlandırılan alt programlardan oluşmuştur.
- C++, Java, JavaScript, JavaApplet, PHP, C# gibi diller C dilinden esinlenmiştir.

# C Dili Taşınabilirdir (Portable) !!!

6

- C dili, **donanımdan bağımsızdır**.
- C dili ile dikkatli bir biçimde yazılmış bir program, her bilgisayara taşınabilir.
  - Yani herhangi bir C programı hiçbir değişikliğe uğramadan, veya çok az bir değişimle, başka bir derleyicide ve/veya işletim sisteminde derlenebilir.
- *Sonuç olarak Windows işletim sistemlerinde yazılan bir C kodu, Linux, UNIX veya VAX gibi işletim sistemlerinde de derlenebilir.*

# C Sistemleri ve Kütüphaneleri

7

- C sistemleri üç kısımdan oluşur:
  1. Programlama ortamı
  2. C programlama dili
  3. C standart kütüphaneleri
- C programları «fonksiyon» adı verilen parçalardan ya da modüllerden oluşur.
- Fonksiyonlar C «bloklarından» oluşur.
- Her fonksiyon/blok bir veya daha fazla «deyimi» içerir.
- Her bir deyim program çalıştırıldığında belirli bir eylemi yerine getirir. Deyimler işlemleri yerine getiren komutlardır.

# C Yazılımı İçeriği

8

Ön işlemci Direktifleri  
(Preprocessor Directives)

Genel Tanımlamalar  
(Global Declarations)

main ( )

{

Yerel Tanımlamalar (Local Declarations)

Deyimler ve İfadeler (Statements)

}

# İlk C Programı

9

```
1  /* Main.c : ilk C programımız */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main ()
6  {
7      printf("Hello World!\n");
8      return 0;
9  }
10
```

- Program çalıştırılması sonucunda ekrana «Hello World!» yazmaktadır.

```
Hello World!
Process returned 0 (0x0)   execution time : 0.005 s
Press any key to continue.
_
```

# 1.Satır: Yorumlar /\* .... \*/ Comments

10

- 1.satır /\* ile başlayıp \*/ ile bitmektedir. Bu işaretler arasına yorum satırları yazılır.
- Yorumlar, okunabilirliği artırmak amacıyla kullandığımız bir özelliktir. Programcı notları olarak da düşünülebilirler.
- Yorum satırlarında bilgisayar hiçbir işlem yapmaz çünkü C derleyicileri bu satırları atlar.
- Dolayısıyla, yorum satırları için makine diline çevrilmiş kodlar oluşturulmaz.

# 1.Satır: Yorumlar /\* .... \*/ Comments (devam...)

11

- Yorumlar, diğer yazılımcıların programınızı anlamasında yardımcı olur ancak çok fazla yorum satırı programın ve kodun okunabilirliğini azaltır.
- Tek satır olan yorumlar için // karakteri de kullanılabilir.
- Yorumlar programın herhangi bir yerinde olabilirler.
- İç içe açıklamalara izin verilmez.
  - /\* yorum deneme /\* .... \*/ .....\*/
- Program başlangıcının; programın yazılma amacı, yazarı, yazılma tarihi vb. yorumlarla olması programlama sürecine adaptasyon noktasında yararlı olabilir.

# Yorum Satırı Örnekleri

12

```
/* Tek satır yorum */
```

```
// Tek satır yorum
```

```
/* İki satır yorum  
örneği */
```

```
/* Birden fazla  
satır yorum  
örneği */
```

```
Toplam = X + Y; // X ve Y toplanarak Toplam değişkenine atanır
```

## 2. ve 3. Satır: Ön işlemci Direktifleri (#include)

13

- # işaretiyle başlayan satırlar, program derlenmeden önce C ön işlemcisi tarafından işlenirler.
- Bu satır, ön işlemciye standart giriş/çıkış **başlık dosyasının** yani **«stdio.h»** dosya içeriğinin programa eklemesini söyler.
- Bu başlık dosyası, derleyicinin **«printf»** gibi standart giriş/çıkış kütüphane fonksiyonlarının (STandarD-Input-Output) derlerken kullanabileceği bilgi ve bildirimleri içerir.
- Başlık dosyalarının uzantısı **.h** dir.

## 2. ve 3. Satır: Ön işlemci Direktifleri (#include) (devam...)

14

- Başlık dosyaları, derleyicinin kütüphane fonksiyonu çağrılarının doğru yapıp yapılmadığını anlamasında yardımcı olan bilgiler içerir.
- ANSI C'deki standart başlık dosyaları şunlardır:

❖ assert.h

❖ ctype.h

❖ errno.h

❖ float.h

❖ limits.h

❖ locale.h

❖ math.h

❖ setjmp.h

❖ signal.h

❖ stdarg.h

❖ stddef.h

❖ stdio.h

❖ stdlib.h

❖ string.h

❖ time.h

## 2. ve 3. Satır: Ön işlemci Direktifleri (#include) (devam...)

15

- «stdio.h» başlık dosyasının eklenmesi tercihe bağlıdır fakat standart giriş/çıkış fonksiyonlarının kullanıldığı programlara eklenmelidir.
- Bu sayede, derleyici, hataları derleme anında bulabilecektir.
- Aksi takdirde, hatalar programın çalıştırıldığı anda ortaya çıkar. Bu tür hataların düzeltilmesi oldukça güç olur.

## 5. Satır: **main( )** fonksiyonu

16

- C programlarının ana fonksiyonu olarak tabir edilir.
- **main** kelimesinden sonraki parantezler ( ) main'in fonksiyon adı verilen program oluşturma bloklarından biri olduğunu gösterir.
- Programın yürütülmesi ilk olarak bu fonksiyonun çağrılmasıyla gerçekleşir.
- C programları bir veya birden fazla fonksiyon içerebilir ancak bunlardan biri mutlaka **main** olmalıdır.

## 5. Satır: **main( )** fonksiyonu (devam...)

17

- Küme parantezi, **{** , her fonksiyonun gövdesinin başına yazılır.
- Diğer küme parantezi , **}** , ise her fonksiyonun sonuna yazılmalıdır.
- Bu iki parantez arasında kalan program parçasığına *«blok»* denir.
- Bloklar C'de önemli program birimleridir.

## 7. Satır: `printf("Hello World!\n");` fonksiyonu

18

- `printf` standart kütüphanede bulunan ekrana formatlı bilgi yazdırma fonksiyondur. Çift tırnak işareti arasındaki karakterleri ekrana yazdırır.
- Yazdırılacak karakterlerin tümüne karakter dizesi «string», mesaj ya da hazır bilgi «literal» denir.
- `printf`, parantezler içindeki bağımsız değişkenler (argument) ve noktalı virgülden oluşan bu satıra «ifade» denir.
- Her ifade ; (noktalı virgül) ile bitmelidir. Noktalı virgüle ifade sonlandırıcı da denir.

## 7. Satır: `printf("Hello World!\n");` fonksiyonu (devam...)

19

- `printf` ifadesindeki tırnak işaretleri arasındaki karakterler aynen ekrana yazdırılır. Ancak `\n` karakterlerinin yazdırılmamaktadır.
- Ters eğik çizgi ( `\` ), **çıkış karakteri** olarak adlandırılır ve `printf`'in farklı bir iş yapması gerektiğini belirtir.
- `printf`, ters çizgi işaretiyle karşılaştığında, bu işareten sonraki karaktere bakar ve bu karaktere göre bazı özel işler yapar.
- Ters çizgi işareti ( `\` ) ve bu işareten sonra gelen karaktere **çıkış sırası** denir.
- `\n` çıkış sırası, **yeni satır anlamına gelir** ve imlecin yeni satıra geçmesine sebep olur.

## 7. Satır: **printf("Hello World!\n");** fonksiyonu (devam...)

20

- Çıkış karakterleri (escape sequence) aşağıdaki gibidir:

| Çıkış     | Anlamı                                             |
|-----------|----------------------------------------------------|
| <b>\0</b> | null karakteri temsil eder (sonlandırıcı karakter) |
| <b>\n</b> | Yeni satır                                         |
| <b>\r</b> | Satırbaşı                                          |
| <b>\t</b> | Yatay sekme                                        |
| <b>\v</b> | Düşey sekme                                        |
| <b>\f</b> | İleri besleme                                      |
| <b>\b</b> | Geri boşluk (space)                                |
| <b>\a</b> | Alarm karakteri                                    |
| <b>\"</b> | Çift tırnak                                        |
| <b>\\</b> | Ters bölü                                          |

## 7. Satır: `printf("Hello World!\n");` fonksiyonu (devam...)

21

```
1  /* Main.c : ilk C programımız */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main ()
6  {
7      printf("1.Hello World!\n");
8      printf("2.Hello ");
9      printf("World!\n");
10     printf("\n");
11     printf("aaaa\t");
12     printf("bbbb\n");
13     printf("cccc\t");
14     printf("dddd\t");
15     printf("\n");
16     return 0;
17 }
18
```



**Ekran Çıktısı  
Ne Olacak**

```
"C:\D\Akademik\Courses\Algoritma ve Programlama I\Labs\Hafta 2\HelloWor
1.Hello World!
2.Hello World!
aaaa      bbbb
cccc      dddd

Process returned 0 (0x0)   execution time : 0.007 s
Press any key to continue.
```

# C Kodlarının Temel Özellikleri - Özet

22

- Yazılımda kullanılacak olan her fonksiyon için ilgili **başlık dosyası** programın başına ilave edilmelidir.
- Her C programı **main()** fonksiyonunu içermelidir.
- Program içinde kullanılacak olan değişkenler ve sabitler mutlaka tanımlanmalıdır.
- Her ifade satırının sonuna **;** işareti konmalıdır.

# C Kodlarının Temel Özellikleri – Özet (devam...)

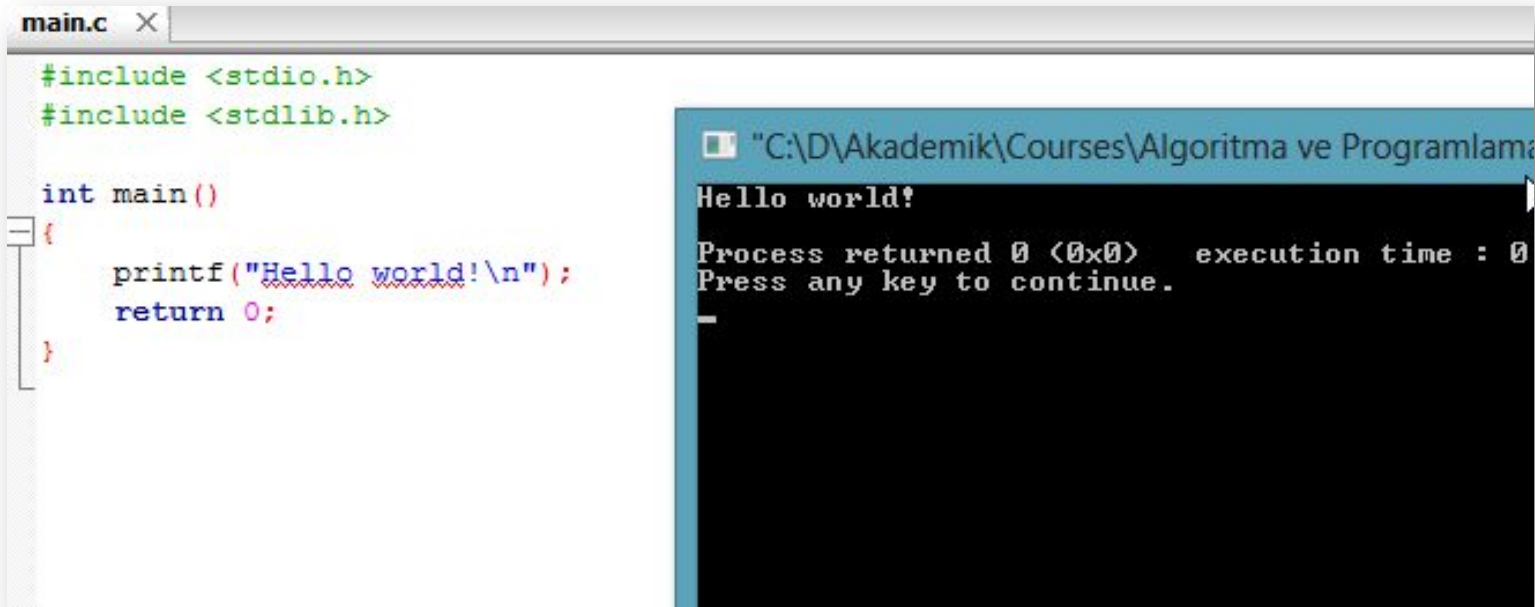
23

- Her bloğun ve fonksiyonun başlangıcı ve bitişi sırasıyla { ve } sembolleridir.
- C dilinde yazılan kodlarda küçük-büyük harf ayrımı vardır (case sensitive).
  - Örneğin; “A” ile “a” derleyici tarafından farklı değerlendirilir.
  - Ogr ogr
- Yorum satırı operatörü /\* \*/ veya // sembolleridir.

# Code::Blocks ile İlk C Programı (devam...)

24

- Son olarak derleme ve çalıştırma işlemi için “F9” kısa yolu veya “Build ☐ Build and run” menü adımı seçilir.



The screenshot displays the Code::Blocks IDE interface. On the left, the 'main.c' editor window shows the following C code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("Hello world!\n");
    return 0;
}
```

On the right, the 'Output' window shows the execution results:

```
"C:\D\Akademik\Courses\Algoritma ve Programlama
Hello world!
Process returned 0 (0x0)   execution time : 0
Press any key to continue.
-
```

# C Programlama Dili Elemanları

25

1. Tanımlayıcılar
2. Anahtar Sözcükler
3. Veri Türleri
4. Değişkenler
5. Sabitler
6. Operatörler

# C Tanımlayıcıları

26

- Programcı tarafından oluşturulurlar.
- Programdaki **değişkenleri**, **sabitleri**, **kayıt alanlarını**, **özel bilgi tiplerini** vb. adlandırmak için kullanılan kelimelerdir.
- Tanımlayıcılar, yerini tuttukları ifadelere çağrışım yapacak şekilde seçilmelidir.
- İngiliz alfabesindeki A-Z veya a-z arasındaki 26 harf ile 0-9 arası rakamlar kullanılabilir.
- Sembollerden sadece alt çizgi **\_** kullanılabilir.
- Tanımlayıcı isimleri **harfle** veya **alt çizgiyle** başlayabilir.
- Tanımlayıcı ismi, **rakamla başlayamaz** veya **sadece rakamlardan oluşamaz**.
- Tanımlayıcılar boşluk karakterini içeremezler.

# C Anahtar Sözcükleri

27

- C dilinde 32 adet anahtar sözcük vardır ve hepsi küçük harfle yazılır. Anahtar sözcükler tanımlayıcı olarak kullanılamazlar.

| <u>Veri tipi</u> | <u>Bellek sınıfı</u> | <u>Deyim</u> | <u>İşleç</u> |
|------------------|----------------------|--------------|--------------|
| char             | auto                 | break        | sizeof       |
| const            | extern               | case         |              |
| double           | register             | continue     |              |
| enum             | static               | default      |              |
| float            | typedef              | do           |              |
| int              |                      | else         |              |
| long             |                      | for          |              |
| short            |                      | goto         |              |
| signed           |                      | if           |              |
| struct           |                      | return       |              |
| union            |                      | switch       |              |
| unsigned         |                      | while        |              |
| void             |                      |              |              |
| volatile         |                      |              |              |

# C Veri Türleri

28

- Veri tipi (data type) program içinde kullanılacak değişken, sabit, fonksiyon isimleri gibi tanımlayıcıların tipini, **yani bellekte ayrılacak bölgenin büyüklüğünü**, belirlemek için kullanılır.
- Bir programcı, bir programlama dilinde ilk olarak öğrenmesi gereken, o dile ait veri tipleridir. Çünkü bu, programcının kullanacağı değişkenlerin ve sabitlerin sınırlarını belirler.

# C Veri Türleri (devam...)

29

- C programlama dilinde 5 tane temel veri tipi bulunmaktadır.
  1. **char**: karakter veriler
  2. **int**: tamsayı veriler
  3. **float**: tek duyarlılıklı kayan noktalı sayılar
  4. **double**: Çift duyarlılıklı kayan noktalı sayılar
  5. **void**: Değer içermeyen verilerdir.

# C Veri Türleri (devam...)

30

- Bazı özel niteleyiciler temel tiplerin önüne gelerek onların türevlerini oluşturur:
  - short
  - long
  - unsigned
- Niteleyiciler değişkenin bellekte kaplayacağı alanı değiştirilebilirler.
- Kısa (**short**), uzun (**long**), ve **normal** (int) tamsayı arasında yalnızca uzunluk farkı vardır. Eğer normal tamsayı 32 bit (4 bayt) ise uzun tamsayı 64 bit (8 bayt) uzunluğunda ve kısa tamsayı 16 biti (2 bayt) geçmeyecek uzunluktadır.

# C Veri Türleri (devam...)

31

- İşaretsiz (**unsigned**) ön eki kullanıldığı takdirde, veri tipi ile saklanacak değerin sıfır ve sıfırdan büyük olması sağlanır. **İşaretli ve işaretsiz verilerin bellekteki uzunlukları aynıdır.** Fakat, işaretsiz tipindeki verilerin üst limiti, işaretlinin iki katıdır.
- Kısa ve uzun tamsayı tutacak tanımlayıcılar için **int** anahtar kelimesinin yazılmasına gerek yoktur.
  - `short s; /* short int s; anlamında */`
  - `long k; /* long int k; anlamında */`

## C Veri Türleri (devam...)

| Veri Tipi          | Açıklama                                   | Bellekte işgal ettiği boyut (bayt) | Alt sınır                  | Üst sınır                  |
|--------------------|--------------------------------------------|------------------------------------|----------------------------|----------------------------|
| char               | Tek bir karakter veya küçük tamsayı için   | 1                                  | -128                       | 127                        |
| unsigned char      |                                            |                                    | 0                          | 255                        |
| short int          | Kısa tamsayı için                          | 2                                  | -32,768                    | 32,767                     |
| unsigned short int |                                            |                                    | 0                          | 65,535                     |
| int                | Tamsayı için                               | 4                                  | -2,147,483,648             | 2,147,483,647              |
| unsigned int       |                                            |                                    | 0                          | 4,294,967,295              |
| long int           | Uzun tamsayı için                          | 8                                  | -9,223,372,036,854,775,808 | 9,223,372,036,854,775,807  |
| unsigned long int  |                                            |                                    | 0                          | 18,446,744,073,709,551,615 |
| float              | Tek duyarlı gerçel sayı için (7 basamak)   | 4                                  | -3.4e +/- 38               | +3.4e +/- 38               |
| double             | Çift duyarlı gerçel sayı için (15 basamak) | 8                                  | -1.7e +/- 308              | +1.7e +/- 308              |

# C Veri Türleri (devam...)

33

```
/*
*****
* Veri türlerinin bellekte kapladığı alanı *
* bulmak için yazılan C programıdır.      *
*****
*/
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf( "char          : %d bayt\n", sizeof(char));
    printf( "short         : %d bayt\n", sizeof(short));
    printf( "int            : %d bayt\n", sizeof(int));
    printf( "long           : %d bayt\n", sizeof(long));
    printf( "unsigned char   : %d bayt\n", sizeof(unsigned char));
    printf( "unsigned short  : %d bayt\n", sizeof(unsigned short));
    printf( "unsigned int    : %d bayt\n", sizeof(unsigned int));
    printf( "unsigned long   : %d bayt\n", sizeof(unsigned long));
    printf( "float          : %d bayt\n", sizeof(float));
    printf( "double         : %d bayt\n", sizeof(double));
    printf( "long double    : %d bayt\n", sizeof(long double));
    return 0;
}
```

"C:\D\Akademik\Courses\Algoritma ve Pro

|                |           |
|----------------|-----------|
| char           | : 1 bayt  |
| short          | : 2 bayt  |
| int            | : 4 bayt  |
| long           | : 4 bayt  |
| unsigned char  | : 1 bayt  |
| unsigned short | : 2 bayt  |
| unsigned int   | : 4 bayt  |
| unsigned long  | : 4 bayt  |
| float          | : 4 bayt  |
| double         | : 8 bayt  |
| long double    | : 12 bayt |

Process returned 0 (0x0) execution t  
Press any key to continue

# C Değişkenleri

34

- Değişken, program içinde kullanılan değerlere bellek üzerinde açılan alanlardır. Bu alanlar bir değişken ismi ile anılır.
- Değişken **isimlendirilmeleri**, tanımlayıcı kurallarına uygun biçimde yapılmalıdır.
- C’de tüm değişkenler kullanılmadan önce programa bildirilmelidir.
- Bu bildirim esnasında, değişkenin veri türü belirlenir.
- Örnek:

**veri\_türü** **değişken\_adı**;

**int** **sayac**;

# C Değişkenleri (devam...)

35

| Değişken/Sabit/<br>Fonksiyon/Yapı Adı | Geçerlilik | Açıklama                                 |
|---------------------------------------|------------|------------------------------------------|
| asal                                  | geçerli    | -                                        |
| Momentum                              | geçerli    | -                                        |
| ivme                                  | geçerli    | -                                        |
| olasilik                              | geçerli    | -                                        |
| IsikHizi                              | geçerli    | -                                        |
| isik_hizi                             | geçerli    | Alt çizgi karakteri '_' kullanılabilir   |
| isik hizi                             | geçersiz   | Boşluk karakteri kullanılamaz            |
| ışık_hızı                             | geçersiz   | Türkçe karakter kullanılamaz             |
| 1Bit                                  | geçersiz   | rakam ile başlanamaz                     |
| typedef                               | geçersiz   | Anahtar kelimelerden birisi kullanılamaz |

# C Değişkenleri (devam...)

36

- Örnekler

```
int x;  
int x1, y1, z1;  
long d, d1;  
char c;  
char c1, c2, c3;  
float a;  
float a1, a2, a3;  
int u[3];  
float k[10*20];
```

- Örnekler

```
int x = 1;  
int x1 = 10, y1 = 20, z1 = 30;  
char c = 'a';  
float a = 123.45;
```

# C Sabitleri

37

- Sabit bildirimi, başlangıç değeri verilen değişken bildirimi gibi yapılır.
- Ancak, veri tipinin önüne **const** anahtar sözcüğü konmalıdır.
- Sabit içerikleri **program boyunca değiştirilemez**. Yalnızca kullanılabilir.
- Genellikle, sabit olarak bildirilen değişken isimleri **büyük harflerle**, diğer değişken isimlerinin ise küçük harflerle yazılması (gösterilmesi) C programcıları tarafından **geleneksel** hale gelmiştir.

# C Sabitleri (devam...)

38

- Örnekler:

**const float** PI = 3.142857;

**const double** NOT= 12345.8596235489;

**const int** EOF= -1;

**const char[]** = "devam etmek için bir tuşa basın...";

# printf () - Tip belirleyici (conversion specifier)

39

- **%** işareti ile başlar ve bir veya iki karakterden oluşur (**%d** gibi).
- Ekranaya yazdırılmak istenen değişkenin tipi, **%** işaretinden sonra belirtilir.
- Ayrıca biçim ifadesinin içine, sola - sağa yaslama, noktadan sonra x basamak yaz vb gibi isteklerimizi belirten karakterler de ekleyebiliriz.
- Gerçek sayıların yazdırılmasında, noktadan sonra yazılacak basamak sayısı durumların ifade edilmesi için ve tamsayıların aynı hizada yazdırılması için **nokta operatörü** veya rakamlar kullanılır.
- Aynı şekilde karakter katarlarının sağa ya da sola dayalı yazdırılması için veya bir karakter katarındaki karakterlerin kaç tanesinin yazdırılacağını belirtmek için de yine **nokta**, **eksi** gibi operatörlerin ve rakamların çeşitli kombinasyonları kullanılır.

# printf () - Tip belirleyici (conversion specifier)

(devam...)

40

|             |                                                                                                                 |
|-------------|-----------------------------------------------------------------------------------------------------------------|
| <b>d</b>    | int türden bir ifadeyi onluk sistemde yazar                                                                     |
| <b>ld</b>   | long türden bir ifadeyi onluk sistemde yazar                                                                    |
| <b>o</b>    | unsigned int türden bir ifadeyi sekizlik sistemde yazar                                                         |
| <b>x, X</b> | unsigned int türden bir ifadeyi onaltılık sistemde yazar; x için küçük harfleri, X için büyük harfleri kullanır |
| <b>lx</b>   | unsigned long türden bir ifadeyi onaltılık sistemde yazar                                                       |
| <b>c</b>    | int veya char türden bir ifadeyi karakter olarak yazar                                                          |
| <b>s</b>    | char * türden bir ifadeyi null karakter ile karşılaşınca kadar, ya da duyarlılıkla belirtilen sayı kadar yazar  |
| <b>u</b>    | unsigned int türden bir ifadeyi onluk sistemde yazar                                                            |
| <b>f</b>    | double türden bir ifadeyi yazar                                                                                 |
| <b>lf</b>   | double veya long double türden bir ifadeyi onluk sistemde yazar                                                 |
| <b>e</b>    | gerçek sayıları üstel olarak yazar                                                                              |
| <b>%</b>    | dönüştürülmez, % olarak yazdırılır                                                                              |

# printf () - Tip belirleyici (conversion specifier)

(devam...)

41

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int    a = 2,    b = 10,    c = 50;
    float  f = 1.05, g = 25.5, h = -0.1, yuzde;

    printf("3 tamsayi      : %d %d %d\n", a, b, c);
    printf("3 tamsayi [TAB] : %d \t%d \t%d\n", a, b, c);

    printf("\n");

    printf("3 reel sayi (yanyana) : %f %f %f\n", f, g, h);
    printf("3 reel sayi (altalta) : \n%f\n%f\n%f\n\n", f, g, h);

    yuzde = 220 * 25/100.0;
    printf("220 nin %%25 i %f dir\n", yuzde);
    printf("%f/%f isleminin sonucu = %f\n", g, f, g / f);

    printf("\nprogram sonunda beep sesi cikar...\a");

    return 0;
}
```

# printf () - Tip belirleyici (conversion specifier)

(devam...)

42

```
"C:\D\Akademik\Courses\Algoritma ve Programlama I\Labs\Hafta 2
3 tamsayi : 2 10 50
3 tamsayi [TAB] : 2      10      50

3 reel sayi <yanyana> : 1.050000 25.500000 -0.100000
3 reel sayi <altalta> :
1.050000
25.500000
-0.100000

220 nin %25 i 55.000000 dir
25.500000/1.050000 isleminin sonucu = 24.285715

program sonunda beep sesi cikar...
```

# C Operatörleri

43

- Operatörler, değişkenler veya sabitler üzerinde matematiksel ve karşılaştırma işlemlerini yapan simgelerdir. Yani bir operatör bir veya daha fazla değişken üzerinde işlem yapan semboldür.
- C programlama dilinde 4 tip operatör bulunmaktadır.
  1. Aritmetik Operatörler
  2. Atama Operatörleri
  3. Karşılaştırma Operatörleri
  4. Mantıksal Operatörler

# C Operatörleri – Aritmetik Operatörler

44

| Operatör | Açıklama | Örnek    | Anlamı               |
|----------|----------|----------|----------------------|
| +        | toplama  | $x + y$  | x ve y nin toplamı   |
| -        | çıkarma  | $x - y$  | x ve y nin farkı     |
| *        | carpma   | $x * y$  | x ve y nin çarpımı   |
| /        | bölme    | $x / y$  | x ve y nin oranı     |
| %        | mod alma | $x \% y$ | x / y den kalan sayı |

- **Örnekler:**

$a = b + 10;$

$c = d + c * e - f / g + h \% j;$

$z = u[1] * u[2];$

$x = 10;$

$a = b = c = 0;$

# C Operatörleri – Atama Operatörleri

45

- Bu operatörler bir değişkene, bir sabit veya bir aritmetik ifade atamak (eşitlemek) için kullanılır.
- Birleşik atama: Bazı ifadelerde işlem operatörü ile atama operatörü birlikte kullanılarak, ifadeler daha kısa yazılabilir.

Eğer ifade

*değişken = değişken [operatör] aritmetik ifade;*

şeklinde ise, daha kısa bir biçimde

*değişken [operatör] = aritmetik ifade;*

olarak yazılabilir.

# C Operatörleri – Atama Operatörleri (devam...)

46

| Operatör | Açıklama              | Örnek        | Anlamı    |
|----------|-----------------------|--------------|-----------|
| =        | atama                 | x = 7;       | x = 7;    |
| +=       | ekleyerek atama       | x += 3       | x = x + 3 |
| -=       | eksilterek atama      | x -= 5       | x = x - 5 |
| *=       | çarparak atama        | x *= 4       | x = x * 4 |
| /=       | bölerek atama         | x /= 2       | x = x / 2 |
| %=       | bölüp, kalanını atama | x %= 9       | x = x % 9 |
| ++       | bir arttırma          | x++ veya ++x | x = x + 1 |
| --       | bir azaltma           | x-- veya --x | x = x - 1 |

op1 işlem = op2

op1 = op1 işlem op 2

# C Operatörleri – Atama Operatörleri (devam...)

47

| Örnek      | Anlamı                                                                             |
|------------|------------------------------------------------------------------------------------|
| $x = y++;$ | y'nin değeri önce x'e aktarılır sonra bir arttırılır.<br>$x = y;$<br>$y = y + 1;$  |
| $x = ++y;$ | y'nin değeri önce bir arttırılır sonra x'e aktarılır .<br>$y = y + 1;$<br>$x = y;$ |
| $x = y--;$ | y'nin değeri önce x'e aktarılır sonra bir azaltılır.<br>$x = y;$<br>$y = y - 1;$   |
| $x = --y;$ | y'nin değeri önce bir azaltılır sonra x'e aktarılır .<br>$y = y - 1;$<br>$x = y;$  |

# C Operatörleri – Atama Operatörleri (devam...)

48

- **Örnek:** Aşağıdaki işlemlerden sonra a, b ve c'nin son değerleri ne olur?

a = 5;

b = a++;

c = ++a;



a = 7

b = 5

c = 7

# C Operatörleri – Atama Operatörleri (devam...)

49

- **Örnek:** Aşağıdaki işlemlerden sonra i'nin son değeri ne olur?

```
int i = 1;
```

```
i++;
```

```
++i;
```

```
i += 1 + i++;
```

```
i = i + 1;
```



i = 9

# scanf() Fonksiyonu

50

- Birçok programda ekrana verilerin bastırılmasının yanı sıra **klavyeden veri okunması** gerekebilir.
- scanf() fonksiyonu klavyeden veri okumak için kullanılan fonksiyondur.
- Tip belirleyicileri printf fonksiyonu ile aynı mantıkta kullanılır ve % sembolü ile ifade edilir.
- Örneğin klavyeden bir x tamsayısı okumak için aşağıdaki ifade kullanılır:  

```
scanf("%d", &x);
```

- `#include <stdio.h>`
- `main()`
- 
- 
- `{ int a,b;`
- `float sonuc;`
- `printf(" iki Deger giriniz");scanf("%d %d", &a,&b);`
- `sonuc= (float)a/b;`
- `printf("\n islem sonucu =% 16.2f",sonuc );`
- 
- `}`

# KAYNAKLAR

52

- Okt. Tuna GÖKSU Bilgisayar ve Programlama Sunumu
- N. Ercil Çağıltay ve ark., C DERSİ PROGRAMLAMAYA GİRİŞ, Ada Matbaacılık, ANKARA; 2009.
- Milli Eğitim Bakanlığı "Programlamaya Giriş ve Algoritmalar Ders Notları", 2007
- <http://tr.wikipedia.org/wiki/Code::Blocks>
- <http://www.codeblocks.org>
- <http://www.AlgoritmaveProgramlama.com>
- <http://www1.gantep.edu.tr/~bingul/c>



Algoritma ve Programlama